SHRI RAMSWAROOP MEMORIAL UNIVERSITY



MongoDB Assignment

UCS2808

SUBMITTED TO:-

ANAND KUMAR

SUBMITTED BY:-

NAME: MUKESH -PRAJAPATI

Roll-Number:202410101230007

Course: B.TECH (CSE)

MongoDB ASSIGNMENT

5 Marker Questions

1 .Explain how you can optimize the performance of an aggregation pipeline considering data size and indexing and discuss the potential limitations of using the aggregation pipeline compared to traditional queries.

Optimizing Aggregation Pipeline Performance:

1. **Use Indexes Efficiently:** Place $match and $sort stages as early as possible in the pipeline so MongoDB can utilize indexes. This reduces the number of documents processed.
2. **Limit Data Early:** Apply $match, $limit, and $project early to reduce the size and number of fields being passed through the pipeline stages.
3. **Avoid Large $lookup:** Minimize the use of $lookup on large collections or use indexed fields for efficient joins to reduce memory usage and latency.
4. **Use $project to Exclude Unneeded Fields:** This reduces the document size in each stage, improving processing speed.

5. **Use allowDiskUse:** For memory-intensive pipelines, enable allowDiskUse to let MongoDB use temporary disk space, preventing memory overflow.

Limitations Compared to Traditional Queries:

- **Performance Overhead:** Aggregation pipelines can be slower for simple queries compared to traditional find() operations.
- **Memory Constraints:** Without allowDiskUse, large pipelines may exceed memory limits.
- **Complexity:** Aggregation syntax is more complex and harder to maintain for simple tasks.
- **Index Use Limitation:** Not all pipeline stages can leverage indexes, unlike traditional queries.

2. Explain how partial indexes can benefit specific query patterns, providing an example. Consider the trade-offs between indexing performance and storage overhead. How would you decide on an effective strategy?

Benefits of Partial Indexes with Example:

Partial indexes index only the documents that meet a specified condition, which can improve performance and reduce storage.

Example: Consider a collection orders where only the delivered orders (status: "delivered") are frequently queried:

db.orders.createIndex({ customer_id: 1 }, { partialFilterExpression: { status: "delivered" } })

This index only includes documents where status = "delivered", making queries like:

db.orders.find({ status: "delivered", customer_id: 123 })

faster and more efficient.

Trade-offs:

- **Pros:**
  - Reduces index size → less disk space and better performance.
  - Faster writes than full indexes since fewer documents are indexed.
  - More relevant to specific query patterns.
- **Cons:**
  - Only benefits queries that match the filter condition.
  - Requires careful query planning to avoid index misses.
  - Additional complexity in index management.

Effective Strategy:

1. **Analyze Query Patterns:** Identify fields that are queried often with specific conditions.
2. **Check Data Distribution:** Use partial indexes when a condition filters out a large portion of documents.
3. **Test Performance:** Benchmark with and without the partial index.
4. **Combine with Monitoring Tools:** Use MongoDB's explain() to ensure queries are using the intended index.

10 Marker Questions

1. Compare and contrast using the $lookup and $unwind operators within the aggregation pipeline. When would you choose one over the other? Provide an example scenario for each. Also, Discuss the potential pitfalls of using the aggregation pipeline for real-time data processing. Suggest alternative approaches or strategies to mitigate these challenges.

Comparison of $lookup and $unwind in Aggregation Pipeline

1. $lookup:

- Purpose: Performs **joins** between collections (like SQL joins).
- Output: Adds matched documents from another collection as an **array field**.
- Use Case: Combine related data from two collections.

Example Scenario: Join orders with customers collection to get customer details:

```
{
  $lookup: {
    from: "customers",
    localField: "customer_id",
    foreignField: "_id",
    as: "customer_info"
  }
}
```

2. $unwind:

- Purpose: **Deconstructs arrays** inside documents.
- Output: Creates a **separate document** for each element in the array.
- Use Case: Flatten arrays for individual processing.

Example Scenario: Break down a product list in an order:

{ $unwind: "$products" }

Each product becomes a separate document in the pipeline.

When to Use Which:

- Use **$lookup** when you need to **merge data** from two collections.
- Use **$unwind** when dealing with arrays and need to process each **element individually**.
- Often used **together**: $lookup to join, followed by $unwind to flatten the joined array.

Pitfalls of Using Aggregation Pipeline for Real-Time Data Processing:

1. **Latency Issues:** Complex stages like $lookup and $group can slow down queries.
2. **Memory Limitations:** 100MB RAM limit per stage (unless allowDiskUse is enabled).
3. **No Built-in Real-Time Triggers:** Aggregation is not inherently real-time or event-driven.
4. **High Resource Usage:** Heavy pipelines can affect performance and scalability under high load.

Alternative Strategies & Solutions:

- **Use allowDiskUse:** Prevents memory overflow in large operations.

- **Pre-Aggregation:** Store summary results in a separate collection for quick access.
- **Use Change Streams:** React to data changes in near real-time.
- **External Tools:** Use Kafka, Spark, or other ETL tools for large-scale real-time data processing.
- **Caching Layers (e.g., Redis):** Cache frequently accessed aggregated results to reduce load.

2. Explore the power of the $accumulator operator. Describe how you can define custom aggregation logic within your pipelines using $accumulator and provide a use case example. How can you effectively combine text processing operators like $substr, $concat, and regular expressions within an aggregation pipeline to analyze textual data?

Power of the $accumulator Operator:

The $accumulator operator in MongoDB allows you to define **custom aggregation logic** using JavaScript expressions. It gives more flexibility than built-in accumulators like $sum or $avg, especially for **complex, stateful, or non-standard computations** within a $group stage.

How It Works:

You define four key functions inside $accumulator:

1. **init** – Initializes the state.
2. **accumulate** – Modifies the state with each input.

3. **merge** – Merges two states (used in parallel processing).
4. **finalize** – Produces the final result.

Use Case Example:

Custom average of squares of numbers in a collection:

```
{
  $group: {
    _id: null,
    avgSquare: {
      $accumulator: {
        init: function () { return { sum: 0, count: 0 }; },
        accumulate: function (state, value) {
        state.sum += value * value;
          state.count += 1;
          return state;
        },
        accumulateArgs: ["$value"],
        merge: function (state1, state2) {
        return {
            sum: state1.sum + state2.sum,
            count: state1.count + state2.count
          };
        },
        finalize: function (state) {
          return state.count === 0 ? 0 : state.sum / state.count;
```

```
      },
      lang: "js"
    }
  }
 }
}
```

Text Processing in Aggregation Pipelines:

MongoDB allows powerful **textual data analysis** using operators like $substr, $concat, and regular expressions with $regexMatch, $regexFind, etc.

Combining Text Operators – Example:

Suppose we want to analyze user comments to extract and format key terms:

```
[
 {
  $project: {
   comment: 1,
   startsWithGood: { $regexMatch: { input: "$comment", regex: /^good/i } },
   first10Chars: { $substr: ["$comment", 0, 10] },
   upperCased: { $toUpper: "$comment" },
   combined: { $concat: ["Summary: ", { $substr: ["$comment", 0, 30] }] }
  }
 }
]
```

Explanation:

$regexMatch: Checks if a comment starts with "good".

$substr: Extracts a substring.

$toUpper: Converts the comment to uppercase.

$concat: Merges a static string with part of the comment for a summary.

Conclusion:

$accumulator unlocks powerful, custom logic for complex aggregations.

Text processing operators like $substr, $concat, and regex functions can be combined to extract, clean, and analyze textual data effectively.

Together, they make MongoDB's aggregation pipeline a powerful tool for custom analytics and text mining.

20 Marker Questions

1. Describe the process of monitoring and analyzing index utilization in MongoDB. How can you identify underutilized or inefficient indexes and optimize your indexing strategy accordingly? And Explore the concept of "covering indexes" and their benefits for improving query performance. Provide an example scenario where a covering index can significantly reduce database I/O operations.

Monitoring and Analyzing Index Utilization in MongoDB

1. Using the db.collection.explain() Method

- Use explain("executionStats") on queries to view how MongoDB is using indexes.
- Look for:
    - IXSCAN – Index is used.
    - COLLSCAN – Collection scan, meaning no index used (potential performance issue).
    - nReturned vs. totalDocsExamined – A high ratio indicates inefficient index usage.

2. Using the db.collection.getIndexes()

- Lists all existing indexes on a collection. Useful for manual review.

3. Using the Index Stats Command

- Run:

```
db.collection.aggregate([
  { $indexStats: {} }
])
```

Shows how frequently each index is accessed (accesses.ops).

Identifies unused or underutilized indexes that could be removed.

Identifying Underutilized or Inefficient Indexes

Underutilized Indexes:

Indexes with low or zero accesses.ops are likely unused. They consume space and slow down writes.

Inefficient Indexes:

If COLLSCAN appears frequently in explain(), queries are not benefiting from available indexes. Also, if totalDocsExamined is very high, the index is not selective enough.

Redundant Indexes:

Two indexes covering the same prefix (e.g., { a: 1 } and { a: 1, b: 1 }) – the shorter one may be redundant.

Optimizing Indexing Strategy

Remove Unused Indexes: Based on index stats, drop indexes with zero usage.

Create Compound Indexes: Combine fields commonly queried together to avoid multiple index scans.

Use Indexes for Sorts: Add indexes that match the sort pattern to avoid in-memory sorts.

Match Query Pattern: Indexes should align with frequent query filters and sort orders.

Use TTL Indexes: For time-based data expiration (e.g., logs).

Covering Indexes – Concept and Benefits

What is a Covering Index?

A covering index is an index that contains all the fields required by a query (both in filter and projection). MongoDB can answer the query entirely from the index, avoiding the need to read documents from disk.

Benefits:

Reduces Disk I/O: No need to fetch documents.

Faster Execution: Entire result fetched from the index.

Lower Memory Usage: Less data processed.

Example Scenario:

Collection: orders

{ "_id": 1, "customer_id": 101, "status": "delivered", "total": 250, "date": "2024-04-01" }

db.orders.find(

{ status: "delivered" },

{ customer_id: 1, total: 1, _id: 0 }

)

db.orders.createIndex({ status: 1, customer_id: 1, total: 1 })

2. Create a detailed case study demonstrating how you would leverage various aggregation operators (e.g., $lookup, $unwind, $geoNear) to analyze complex data relationships and perform geospatial queries in a MongoDB application

Case Study: "FoodFinder" – A Restaurant Discovery Application

Objective:

To help users find nearby restaurants that serve a specific cuisine, show reviews from other users, and calculate average ratings—while analyzing the spatial and relational data efficiently using MongoDB's **aggregation pipeline**.

Database Structure:

1. restaurants Collection:

```
{
  "_id": ObjectId("..."),
  "name": "Spice Hub",
  "location": { "type": "Point", "coordinates": [77.5946, 12.9716] }, // [lng, lat]
  "cuisine": "Indian"
}
```

reviews Collection:

{

```
  "_id": ObjectId("..."),
  "restaurant_id": ObjectId("..."),
  "user_id": ObjectId("..."),
  "rating": 4,
  "comment": "Great food!"
}
users Collection:
{
  "_id": ObjectId("..."),
  "username": "foodie123",
  "location": { "type": "Point", "coordinates": [77.5980, 12.9721] }
}
```

Aggregation Pipeline Breakdown:

```
db.restaurants.aggregate([
  // 1. Geospatial query to find nearby restaurants
  {
    $geoNear: {
      near: { type: "Point", coordinates: [77.5980, 12.9721] }, // User's location
      distanceField: "distance",
      maxDistance: 5000,
      spherical: true,
      query: { cuisine: "Indian" }
    }
  },
```

```
// 2. Lookup reviews for each restaurant
{
  $lookup: {
    from: "reviews",
    localField: "_id",
    foreignField: "restaurant_id",
    as: "reviews"
  }
},

// 3. Unwind the reviews to process them individually
{
  $unwind: {
    path: "$reviews",
    preserveNullAndEmptyArrays: true
  }
},

// 4. Lookup user info for each review
{
  $lookup: {
    from: "users",
    localField: "reviews.user_id",
    foreignField: "_id",
    as: "reviewer"
```

```
  }
},

// 5. Simplify the nested reviewer array
{
  $unwind: {
    path: "$reviewer",
    preserveNullAndEmptyArrays: true
  }
},

// 6. Group by restaurant to get average rating and embed reviews
{
  $group: {
    _id: "$_id",
    name: { $first: "$name" },
    cuisine: { $first: "$cuisine" },
    distance: { $first: "$distance" },
    location: { $first: "$location" },
    averageRating: { $avg: "$reviews.rating" },
    reviews: {
      $push: {
        rating: "$reviews.rating",
        comment: "$reviews.comment",
        reviewer: "$reviewer.username"
      }
```

```
    }
  }
 },

 // 7. Sort by rating
 { $sort: { averageRating: -1 } }
])
```

Results:

- Returns a list of **nearby Indian restaurants**.
- Each entry includes:
  - Restaurant details
  - Distance from the user
  - **Average rating**
  - Reviews with **usernames** and comments
- Sorted by highest-rated places.

Operators Used & Their Purpose:

Operator	Purpose
$geoNear	Finds restaurants close to the user based on geospatial location.
$lookup	Joins reviews and user info into restaurant results.
$unwind	Flattens arrays for processing each review and user separately.
$group	Aggregates reviews per restaurant and calculates the average rating.
$sort	Orders results by rating for best user experience.

Benefits of This Approach:

1. **Real-Time Location-Aware Discovery:** Efficient geospatial filtering using $geoNear.
2. **Complex Relationship Analysis:** Combines multiple related collections (reviews, users).
3. **Scalable Aggregation:** Designed to scale by filtering early and grouping late.
4. **Custom Metrics:** Enables on-the-fly computations like average ratings.